

```
In [1]: import pandas as pd
import numpy as np
```

```
/opt/anaconda3/lib/python3.13/site-packages/pandas/core/computation/expressions.py:22: UserWarning: Pandas requires version '2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
```

Loading the CSV files

```
In [2]: ratings_df = pd.read_csv("./ratings.csv")
```

```
In [3]: reviews_df = pd.read_csv("./reviews.csv")
```

```
In [4]: subscribers_df = pd.read_csv("./subscribers.csv")
```

```
In [5]: titles_df = pd.read_csv("./titles.csv")
```

```
In [6]: watch_history_df = pd.read_csv("./watch_history.csv")
```

```
In [7]: watchlist_df = pd.read_csv("./watchlist.csv")
```

Running Basic Profile Test for all the files

```
In [8]: def runnning_basic_profile(df, name="data_drame"):
    print(f'{'-' * 50} {name} {'-' * 50}')
```

getting the columns of the dataframe

```
    cols = list(df.columns)

    # printing the number of rows in csv file
    print(f'The row count in {name} is: {len(df)}')
```

printing the number of columns in csv file

```
    print(f'{'*' * 100}')
```

printing the number of columns in csv file

```
    print(f'The number of columns in {name} is: {len(cols)}')
```

printing the datatypes for every column in the df

```
    for col in cols:

        print(f'The data type for {col} column in {name} is: {df[col].dtype}')
```

```
In [9]: runnning_basic_profile(ratings_df, "ratings.csv")
```

```
----- ratings.csv -----  
-----  
The row count in ratings.csv is: 130000  
*****  
*****  
The number of columns in ratings.csv is: 5  
*****  
*****  
The data type for rating_id column in ratings.csv is: int64  
The data type for subscriber_id column in ratings.csv is: str  
The data type for title_id column in ratings.csv is: str  
The data type for rating column in ratings.csv is: int64  
The data type for rating_date column in ratings.csv is: str
```

```
In [10]: runnning_basic_profile(reviews_df, "reviews.csv")
```

```
----- reviews.csv -----  
-----  
The row count in reviews.csv is: 110000  
*****  
*****  
The number of columns in reviews.csv is: 7  
*****  
*****  
The data type for review_id column in reviews.csv is: int64  
The data type for subscriber_id column in reviews.csv is: str  
The data type for title_id column in reviews.csv is: str  
The data type for review_text column in reviews.csv is: str  
The data type for sentiment column in reviews.csv is: str  
The data type for helpful_votes column in reviews.csv is: int64  
The data type for review_date column in reviews.csv is: str
```

```
In [11]: runnning_basic_profile(subscribers_df, "subscribers.csv")
```

```
----- subscribers.csv -----  
-----  
The row count in subscribers.csv is: 15000  
*****  
*****  
The number of columns in subscribers.csv is: 14  
*****  
*****  
The data type for subscriber_id column in subscribers.csv is: str  
The data type for signup_date column in subscribers.csv is: str  
The data type for country column in subscribers.csv is: str  
The data type for region column in subscribers.csv is: str  
The data type for age column in subscribers.csv is: int64  
The data type for gender column in subscribers.csv is: str  
The data type for plan_type column in subscribers.csv is: str  
The data type for monthly_price_usd column in subscribers.csv is: float64  
The data type for household_size column in subscribers.csv is: int64  
The data type for primary_device column in subscribers.csv is: str  
The data type for payment_method column in subscribers.csv is: str  
The data type for tenure_months column in subscribers.csv is: int64  
The data type for is_active column in subscribers.csv is: bool  
The data type for churn_date column in subscribers.csv is: str
```

```
In [12]: runnning_basic_profile(titles_df, "titles.csv")
```

```

----- titles.csv -----
-----
The row count in titles.csv is: 9000
*****
*****
The number of columns in titles.csv is: 21
*****
*****
The data type for title_id column in titles.csv is: str
The data type for title_name column in titles.csv is: str
The data type for type column in titles.csv is: str
The data type for primary_genre column in titles.csv is: str
The data type for country column in titles.csv is: str
The data type for language column in titles.csv is: str
The data type for release_year column in titles.csv is: int64
The data type for date_added column in titles.csv is: str
The data type for maturity_rating column in titles.csv is: str
The data type for seasons column in titles.csv is: int64
The data type for content_duration_min column in titles.csv is: int64
The data type for is_original column in titles.csv is: bool
The data type for license_type column in titles.csv is: str
The data type for director column in titles.csv is: str
The data type for cast column in titles.csv is: str
The data type for quality_score column in titles.csv is: float64
The data type for popularity_score column in titles.csv is: float64
The data type for license_cost_usd column in titles.csv is: int64
The data type for license_expiry column in titles.csv is: str
The data type for total_watch_hours column in titles.csv is: float64
The data type for total_plays column in titles.csv is: int64

```

```
In [13]: runnning_basic_profile(watch_history_df, "watch_history.csv")
```

```

----- watch_history.csv -----
-----
The row count in watch_history.csv is: 650000
*****
*****
The number of columns in watch_history.csv is: 10
*****
*****
The data type for watch_id column in watch_history.csv is: int64
The data type for subscriber_id column in watch_history.csv is: str
The data type for title_id column in watch_history.csv is: str
The data type for watch_date column in watch_history.csv is: str
The data type for device column in watch_history.csv is: str
The data type for region column in watch_history.csv is: str
The data type for content_duration_min column in watch_history.csv is: int
64
The data type for watch_duration_min column in watch_history.csv is: float
64
The data type for completion_pct column in watch_history.csv is: float64
The data type for completed column in watch_history.csv is: bool

```

```
In [14]: runnning_basic_profile(watchlist_df, "watchlist.csv")
```

```
----- watchlist.csv -----
-----
The row count in watchlist.csv is: 65000
*****
*****
The number of columns in watchlist.csv is: 5
*****
*****
The data type for watchlist_id column in watchlist.csv is: int64
The data type for subscriber_id column in watchlist.csv is: str
The data type for title_id column in watchlist.csv is: str
The data type for added_date column in watchlist.csv is: str
The data type for watched column in watchlist.csv is: bool
```

Cheking for missing values in every column for every file

```
In [15]: def checking_missing_values(df, name="data_frame"):
          print(f'{'-' * 50} {name} {'-' * 50}')
          # getting the columns of the dataframe
          cols = list(df.columns)
          null_rows = False

          for col in cols:
              null_values = df[col].isnull().sum()
              if null_values > 0:
                  null_rows = True
                  percentage_rows_affected = round((null_values / len(df)) * 100)
                  print(f'{col} in {name} has {null_values} null values which affects {percentage_rows_affected}% of rows.')

          if null_rows is False:
              print(f'{name} has 0 null values.')
```

```
In [16]: checking_missing_values(ratings_df, "ratings.csv")
```

```
----- ratings.csv -----
-----
ratings.csv has 0 null values.
```

```
In [17]: checking_missing_values(reviews_df, "reviews.csv")
```

```
----- reviews.csv -----
-----
reviews.csv has 0 null values.
```

```
In [18]: checking_missing_values(subscribers_df, "subscribers.csv")
```

```
----- subscribers.csv -----
-----
churn_date in subscribers.csv has 11199 null values which affects 74.66% of rows.
```

```
In [19]: checking_missing_values(titles_df, "titles.csv")
```

```
----- titles.csv -----
-----
license_expiry in titles.csv has 1966 null values which affects 21.84% of
rows.
```

```
In [20]: checking_missing_values(watch_history_df, "watch_history.csv")
```

```
----- watch_history.csv -----
-----
watch_history.csv has 0 null values.
```

```
In [21]: checking_missing_values(watchlist_df, "watchlist.csv")
```

```
----- watchlist.csv -----
-----
watchlist.csv has 0 null values.
```

Checking for duplicate rows

```
In [22]: def checking_duplicate_rows(df, name="data_frame"):
          print(f'{'-' * 50} {name} {'-' * 50}')
          # getting the columns of the dataframe
          cols = list(df.columns)

          contain_duplicate = False

          for col in cols:

              duplicate_rows = df[col].duplicated().sum()

              if duplicate_rows > 0:
                  contain_duplicate = True
                  print(f'{col} in {name} has {duplicate_rows} duplicate rows.'

              if contain_duplicate is False:
                  print(f'{name} does not have any columns which have duplicate row
```

```
In [23]: checking_duplicate_rows(ratings_df, "ratings.csv")
```

```
----- ratings.csv -----
-----
subscriber_id in ratings.csv has 115005 duplicate rows.
title_id in ratings.csv has 121001 duplicate rows.
rating in ratings.csv has 129995 duplicate rows.
rating_date in ratings.csv has 126471 duplicate rows.
```

The primary key in ratings.csv which is rating_id does not have duplicate values therefore do not violate any integrity constraints.

```
In [24]: checking_duplicate_rows(reviews_df, "reviews.csv")
```

```
----- reviews.csv -----  
-----  
subscriber_id in reviews.csv has 95009 duplicate rows.  
title_id in reviews.csv has 101001 duplicate rows.  
review_text in reviews.csv has 109980 duplicate rows.  
sentiment in reviews.csv has 109997 duplicate rows.  
helpful_votes in reviews.csv has 109975 duplicate rows.  
review_date in reviews.csv has 106523 duplicate rows.  
review_text in reviews.csv has 109980 duplicate rows.  
sentiment in reviews.csv has 109997 duplicate rows.  
helpful_votes in reviews.csv has 109975 duplicate rows.  
review_date in reviews.csv has 106523 duplicate rows.
```

The primary key in reviews.csv which is review_id does not have duplicate values therefore do not violate any integrity constraints.

```
In [25]: checking_duplicate_rows(subscribers_df, "subscribers.csv")
```

```
----- subscribers.csv -----  
-----  
signup_date in subscribers.csv has 11565 duplicate rows.  
country in subscribers.csv has 14980 duplicate rows.  
region in subscribers.csv has 14991 duplicate rows.  
age in subscribers.csv has 14930 duplicate rows.  
gender in subscribers.csv has 14996 duplicate rows.  
plan_type in subscribers.csv has 14997 duplicate rows.  
monthly_price_usd in subscribers.csv has 14997 duplicate rows.  
household_size in subscribers.csv has 14996 duplicate rows.  
primary_device in subscribers.csv has 14994 duplicate rows.  
payment_method in subscribers.csv has 14994 duplicate rows.  
tenure_months in subscribers.csv has 14874 duplicate rows.  
is_active in subscribers.csv has 14998 duplicate rows.  
churn_date in subscribers.csv has 13332 duplicate rows.
```

The primary key in subscribers.csv which is subscriber_id does not have duplicate values therefore do not violate any integrity constraints.

```
In [26]: checking_duplicate_rows(titles_df, "titles.csv")
```

```
----- titles.csv -----
-----
title_name in titles.csv has 53 duplicate rows.
type in titles.csv has 8998 duplicate rows.
primary_genre in titles.csv has 8984 duplicate rows.
country in titles.csv has 8980 duplicate rows.
language in titles.csv has 8986 duplicate rows.
release_year in titles.csv has 8942 duplicate rows.
date_added in titles.csv has 5986 duplicate rows.
maturity_rating in titles.csv has 8990 duplicate rows.
seasons in titles.csv has 8988 duplicate rows.
content_duration_min in titles.csv has 8126 duplicate rows.
is_original in titles.csv has 8998 duplicate rows.
license_type in titles.csv has 8996 duplicate rows.
director in titles.csv has 7148 duplicate rows.
quality_score in titles.csv has 8215 duplicate rows.
popularity_score in titles.csv has 8473 duplicate rows.
license_cost_usd in titles.csv has 10 duplicate rows.
license_expiry in titles.csv has 7324 duplicate rows.
total_watch_hours in titles.csv has 4620 duplicate rows.
total_plays in titles.csv has 8651 duplicate rows.
quality_score in titles.csv has 8215 duplicate rows.
popularity_score in titles.csv has 8473 duplicate rows.
license_cost_usd in titles.csv has 10 duplicate rows.
license_expiry in titles.csv has 7324 duplicate rows.
total_watch_hours in titles.csv has 4620 duplicate rows.
total_plays in titles.csv has 8651 duplicate rows.
```

The primary key in titles.csv which is title_id does not have duplicate values therefore do not violate any integrity constraints.

```
In [27]: checking_duplicate_rows(watch_history_df, "watch_history.csv")
```

```
----- watch_history.csv -----
-----
subscriber_id in watch_history.csv has 635002 duplicate rows.
title_id in watch_history.csv has 641000 duplicate rows.
watch_date in watch_history.csv has 647384 duplicate rows.
device in watch_history.csv has 649994 duplicate rows.
region in watch_history.csv has 649991 duplicate rows.
content_duration_min in watch_history.csv has 649126 duplicate rows.
watch_duration_min in watch_history.csv has 626145 duplicate rows.
completion_pct in watch_history.csv has 649009 duplicate rows.
completed in watch_history.csv has 649998 duplicate rows.
```

The primary key in watch_history.csv which is watch_id does not have duplicate values therefore do not violate any integrity constraints.

```
In [28]: checking_duplicate_rows(watchlist_df, "watchlist.csv")
```

----- watchlist.csv -----

```

subscriber_id in watchlist.csv has 50201 duplicate rows.
title_id in watchlist.csv has 56043 duplicate rows.
added_date in watchlist.csv has 61837 duplicate rows.
watched in watchlist.csv has 64998 duplicate rows.
subscriber_id in watchlist.csv has 50201 duplicate rows.
title_id in watchlist.csv has 56043 duplicate rows.
added_date in watchlist.csv has 61837 duplicate rows.
watched in watchlist.csv has 64998 duplicate rows.

```

The primary key in watchlist.csv which is watchlist_id does not have duplicate values therefore do not violate any integrity constraints.

Validating Data Types

-----ratings.csv-----

- rating_date in ratings.csv should be of datetime type.

```

In [29]: # changing rating_date type in ratings_df to datetime type
ratings_df['rating_date'] = pd.to_datetime(ratings_df['rating_date'])
ratings_df.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 130000 entries, 0 to 129999
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   rating_id       130000 non-null   int64
1   subscriber_id   130000 non-null   str
2   title_id        130000 non-null   str
3   rating          130000 non-null   int64
4   rating_date     130000 non-null   datetime64[us]
dtypes: datetime64[us](1), int64(2), str(2)
memory usage: 7.2 MB

```

-----reviews.csv---

- review_date in reviews.csv should be of datetime type.

```

In [30]: # changing review_date type in reviews_df to datetime type
reviews_df['review_date'] = pd.to_datetime(reviews_df['review_date'])
reviews_df.info()

```

```
<class 'pandas.DataFrame'>
RangeIndex: 110000 entries, 0 to 109999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   review_id             110000 non-null  int64
1   subscriber_id         110000 non-null  str
2   title_id              110000 non-null  str
3   review_text           110000 non-null  str
4   sentiment             110000 non-null  str
5   helpful_votes         110000 non-null  int64
6   review_date           110000 non-null  datetime64[us]
dtypes: datetime64[us](1), int64(2), str(4)
memory usage: 12.8 MB
```

----- subscribers.csv----- -----

- signup_date in subscribers.csv should of datetime type.
- churn_date in subscribers.csv should of datetime type.

```
In [31]: # changing signup_date type in subscribers_df to datetime type
subscribers_df['signup_date'] = pd.to_datetime(subscribers_df['signup_date'])
# changing churn_date type in subscribers_df to datetime type
subscribers_df['churn_date'] = pd.to_datetime(subscribers_df['churn_date'])
subscribers_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   subscriber_id         15000 non-null  str
1   signup_date           15000 non-null  datetime64[us]
2   country               15000 non-null  str
3   region               15000 non-null  str
4   age                  15000 non-null  int64
5   gender               15000 non-null  str
6   plan_type            15000 non-null  str
7   monthly_price_usd    15000 non-null  float64
8   household_size        15000 non-null  int64
9   primary_device        15000 non-null  str
10  payment_method        15000 non-null  str
11  tenure_months        15000 non-null  int64
12  is_active             15000 non-null  bool
13  churn_date            3801 non-null   datetime64[us]
dtypes: bool(1), datetime64[us](2), float64(1), int64(3), str(7)
memory usage: 2.4 MB
```

-----titles.csv----- -----

- date_added in titles.csv should be of datetime type.

- is_original in titles.csv should be of boolean type.
- license_cost_usd in titles.csv should be of float type.
- license_expiry in titles.csv should be of datetime type.

```
In [32]: # Changing date_added type in titles_df to datetime type
titles_df['date_added'] = pd.to_datetime(titles_df['date_added'])

# changing is_original type in titles_df to boolean type
titles_df['is_original'] = titles_df['is_original'].astype(bool)

# changing license_cost_usd in titles_df to float type
titles_df['license_cost_usd'] = titles_df['license_cost_usd'].astype(float)

# changing license_expiry type in titles_df to datetime type
titles_df['license_expiry'] = pd.to_datetime(titles_df['license_expiry'])

titles_df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 9000 entries, 0 to 8999
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	title_id	9000 non-null	str
1	title_name	9000 non-null	str
2	type	9000 non-null	str
3	primary_genre	9000 non-null	str
4	country	9000 non-null	str
5	language	9000 non-null	str
6	release_year	9000 non-null	int64
7	date_added	9000 non-null	datetime64[us]
8	maturity_rating	9000 non-null	str
9	seasons	9000 non-null	int64
10	content_duration_min	9000 non-null	int64
11	is_original	9000 non-null	bool
12	license_type	9000 non-null	str
13	director	9000 non-null	str
14	cast	9000 non-null	str
15	quality_score	9000 non-null	float64
16	popularity_score	9000 non-null	float64
17	license_cost_usd	9000 non-null	float64
18	license_expiry	7034 non-null	datetime64[us]
19	total_watch_hours	9000 non-null	float64
20	total_plays	9000 non-null	int64

```
dtypes: bool(1), datetime64[us](2), float64(4), int64(4), str(10)
```

```
memory usage: 2.6 MB
```

----- watch_history.csv----- ---

- watch_date in watch_histroy should be of datetime type.

```
In [33]: # changing watch_date type in watch_history_df to datetime type
watch_history_df['watch_date'] = pd.to_datetime(watch_history_df['watch_d
```

```
watch_history_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 650000 entries, 0 to 649999
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   watch_id                             650000 non-null  int64
1   subscriber_id                       650000 non-null  str
2   title_id                            650000 non-null  str
3   watch_date                          650000 non-null  datetime64[us]
4   device                              650000 non-null  str
5   region                              650000 non-null  str
6   content_duration_min                650000 non-null  int64
7   watch_duration_min                  650000 non-null  float64
8   completion_pct                      650000 non-null  float64
9   completed                           650000 non-null  bool
dtypes: bool(1), datetime64[us](1), float64(2), int64(2), str(4)
memory usage: 67.5 MB
```

```
-----watchlist.csv-----
-----
```

- added_date in watchlist.csv should be of datetime type.

```
In [34]: # changing added_date type in watchlist_df to datetime type
watchlist_df['added_date'] = pd.to_datetime(watchlist_df['added_date'])
watchlist_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 65000 entries, 0 to 64999
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   watchlist_id    65000 non-null  int64
1   subscriber_id   65000 non-null  str
2   title_id        65000 non-null  str
3   added_date      65000 non-null  datetime64[us]
4   watched         65000 non-null  bool
dtypes: bool(1), datetime64[us](1), int64(1), str(2)
memory usage: 3.2 MB
```

Checking for outliers in watch_duration_min

```
In [35]: import numpy as np

watch_duration = watch_history_df['watch_duration_min']
mean = np.mean(watch_duration)
std = np.std(watch_duration)

lower_bound = mean - 4 * std
upper_bound = mean + 4 * std

# Enclose each boolean condition in parentheses
```

```
statistical_outliers = (watch_history_df['watch_duration_min'] < lower_bo
logical_violations = (watch_history_df['watch_duration_min'] > watch_hist

outliers_watch_history = watch_history_df[statistical_outliers | logical_

# Create specific flag columns
watch_history_df['is_stat_outlier'] = (watch_history_df['watch_duration_m
watch_history_df['exceeds_content_len'] = watch_history_df['watch_duratio

# Combined flag
watch_history_df['is_flagged'] = watch_history_df['is_stat_outlier'] | wa

# Filter flagged records
outliers_watch_history = watch_history_df[watch_history_df['is_flagged']]
print(f"Total flagged sessions: {len(outliers_watch_history)}")

print(watch_history_df[watch_history_df['is_flagged'] == True])
```

Total flagged sessions: 5769

	watch_id	subscriber_id	title_id	watch_date	device	\
15	16	SUB100917	TTL208766	2026-05-21	Smart TV	
220	221	SUB109518	TTL205837	2026-01-28	Mobile	
320	321	SUB109615	TTL201079	2026-03-25	Game Console	
357	358	SUB110305	TTL201378	2026-05-04	Smart TV	
440	441	SUB105114	TTL204995	2026-04-21	Smart TV	
...	...	...	...	...	...	...
648902	648903	SUB112135	TTL203399	2025-06-10	Smart TV	
648928	648929	SUB103795	TTL208678	2026-04-26	Smart TV	
648978	648979	SUB101002	TTL204327	2026-05-11	Mobile	
649039	649040	SUB100887	TTL207442	2025-07-18	Game Console	
649911	649912	SUB108612	TTL207270	2026-05-03	Mobile	

	region	content_duration_min	watch_duration_min	\
15	North America	2800	2630.4	
220	South Asia	2832	2223.7	
320	Europe	3036	2078.1	
357	Latin America	2508	2057.6	
440	North America	2574	2376.7	
...	...	...	...	...
648902	East Asia	3456	3393.1	
648928	Latin America	2544	2467.2	
648978	North America	3920	2229.6	
649039	North America	2862	2722.2	
649911	Latin America	3300	2189.1	

	completion_pct	completed	is_stat_outlier	exceeds_content_len	\
15	93.9	True	True	False	
220	78.5	False	True	False	
320	68.4	False	True	False	
357	82.0	False	True	False	
440	92.3	True	True	False	
...	...	...	...	...	...
648902	98.2	True	True	False	
648928	97.0	True	True	False	
648978	56.9	False	True	False	
649039	95.1	True	True	False	
649911	66.3	False	True	False	

	is_flagged
15	True
220	True
320	True
357	True
440	True
...	...
648902	True
648928	True
648978	True
649039	True
649911	True

[5769 rows x 13 columns]

```
In [36]: # From watch_history_df remove the entries where is_flagged = True and st
watch_history_df = watch_history_df[watch_history_df['is_flagged'] == Fal
# checking for null values
```

```
checking_missing_values(watch_history_df, "watch_history.csv")

# from watch_history_df remove columns is_stat_outlier, exceeds_content_l
watch_history_df = watch_history_df.drop(['is_stat_outlier', 'exceeds_con
watch_history_df
```

----- watch_history.csv -----

watch_history.csv has 0 null values.

Out[36]:

	watch_id	subscriber_id	title_id	watch_date	device	region	col
0	1	SUB107630	TTL205207	2024-01-01	Smart TV	East Asia	
1	2	SUB114773	TTL204889	2025-10-18	Mobile	South Asia	
2	3	SUB106013	TTL206882	2026-04-26	Mobile	Latin America	
3	4	SUB100939	TTL208849	2026-03-09	Mobile	North America	
4	5	SUB107223	TTL202529	2023-12-27	Tablet	North America	
...	...	...	...	...	...	...	...
649995	649996	SUB113946	TTL207486	2025-07-13	Mobile	Europe	
649996	649997	SUB108172	TTL200072	2026-05-22	Smart TV	Latin America	
649997	649998	SUB108092	TTL204011	2025-01-25	Streaming Stick	South Asia	
649998	649999	SUB114820	TTL204762	2026-04-13	Tablet	North America	
649999	650000	SUB101592	TTL206202	2022-10-16	Game Console	Latin America	

644231 rows × 10 columns

Verifying Referential Integrity

```
In [37]: def missing_fk(child_df, parent_df, fk_col, pk_col=None, df_name="Child T
        """Identifies foreign key records in child_df that do not exist in pa
        if pk_col is None:
            pk_col = fk_col

        # Identify records in the child table missing from the parent table
        orphan_records = child_df[~child_df[fk_col].isin(parent_df[pk_col])]

        orphan_unique_count = orphan_records[fk_col].nunique()
        total_orphans = len(orphan_records)

        print(f"--- Foreign Key Integrity Check: {df_name} ---")
```

```

print(f"Total orphan records: {total_orphans}")
print(f"Unique missing '{fk_col}' keys: {orphan_unique_count}")

if total_orphans > 0:
    print("\nSample orphan records:")
    print(orphan_records.head())

return orphan_records

```

-----watch_history.csv-----

```

In [38]: missing_fk(
    child_df = watch_history_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Watch History'
)

```

--- Foreign Key Integrity Check: Watch History ---
Total orphan records: 0
Unique missing 'subscriber_id' keys: 0

```

Out[38]:  watch_id  subscriber_id  title_id  watch_date  device  region  content_duration_r

```

```

In [39]: missing_fk(
    child_df = watch_history_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Watch History'
)

```

--- Foreign Key Integrity Check: Watch History ---
Total orphan records: 0
Unique missing 'title_id' keys: 0

```

Out[39]:  watch_id  subscriber_id  title_id  watch_date  device  region  content_duration_r

```

-----ratings.csv-----

```

In [40]: missing_fk(
    child_df = ratings_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Ratings'
)

```

--- Foreign Key Integrity Check: Ratings ---
Total orphan records: 0
Unique missing 'subscriber_id' keys: 0

Out[40]: **rating_id subscriber_id title_id rating rating_date**

```
In [41]: missing_fk(
    child_df = ratings_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Ratings'
)
```

--- Foreign Key Integrity Check: Ratings ---

Total orphan records: 0

Unique missing 'title_id' keys: 0

Out[41]: **rating_id subscriber_id title_id rating rating_date**

-----reviews.csv-----

```
In [42]: missing_fk(
    child_df = reviews_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Reviews'
)
```

--- Foreign Key Integrity Check: Reviews ---

Total orphan records: 0

Unique missing 'subscriber_id' keys: 0

Out[42]: **review_id subscriber_id title_id review_text sentiment helpful_votes review_c**

```
In [43]: missing_fk(
    child_df = reviews_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Reviews'
)
```

--- Foreign Key Integrity Check: Reviews ---

Total orphan records: 0

Unique missing 'title_id' keys: 0

Out[43]: **review_id subscriber_id title_id review_text sentiment helpful_votes review_c**

-----watchlist.csv-----

```
In [44]: missing_fk(
    child_df = watchlist_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
```

```
pk_col = 'subscriber_id',
df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---

Total orphan records: 0

Unique missing 'subscriber_id' keys: 0

Out[44]:

watchlist_id	subscriber_id	title_id	added_date	watched
--------------	---------------	----------	------------	---------

```
In [45]: missing_fk(
    child_df = watchlist_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---

Total orphan records: 0

Unique missing 'title_id' keys: 0

Out[45]:

watchlist_id	subscriber_id	title_id	added_date	watched
--------------	---------------	----------	------------	---------

None of the files break the referential integrity constraints, however if the rows are removed due to some other reasons then we need to run the referential integrity check again.

Running Specific Checks

-----subscribers.csv-----

```
In [46]: # For churned customers confirming that churn_date is after signup_date.
subscribers_df[subscribers_df['churn_date'] < subscribers_df['signup_date']]
```

Out[46]:

subscriber_id	signup_date	country	region	age	gender	plan_type	monthly_pi
---------------	-------------	---------	--------	-----	--------	-----------	------------

```
In [47]: # confirming active subscribers (is_active=True) have a blank churn_date
subscribers_df[(subscribers_df['is_active'] == True) & (subscribers_df['c
```

Out[47]:

subscriber_id	signup_date	country	region	age	gender	plan_type	monthly_pi
---------------	-------------	---------	--------	-----	--------	-----------	------------

```
In [48]: # Are there subscribers with very short tenure and still marked as active
subscribers_df[(subscribers_df['tenure_months'] < 3) & (subscribers_df['i

# creating new boolean column for short_tenure where tenure_months < 3 an
subscribers_df['short_tenure'] = (subscribers_df['tenure_months'] < 3) &

subscribers_df[subscribers_df['short_tenure'] == True]
```

Out [48]:

	subscriber_id	signup_date	country	region	age	gender	plan_type	mo
66	SUB100066	2026-05-18	Japan	East Asia	25	Male	Basic with Ads	
87	SUB100087	2026-03-29	South Korea	East Asia	13	Male	Standard	
102	SUB100102	2026-04-30	India	South Asia	22	Male	Standard	
144	SUB100144	2026-04-10	South Korea	East Asia	32	Male	Premium	
170	SUB100170	2026-04-14	India	South Asia	28	Male	Premium	
...	...	...	...	...	...	...	...	...
14943	SUB114943	2026-04-29	United States	North America	55	Female	Premium	
14947	SUB114947	2026-03-09	South Korea	East Asia	25	Female	Standard	
14964	SUB114964	2026-03-09	Japan	East Asia	29	Male	Basic with Ads	
14987	SUB114987	2026-03-22	India	South Asia	49	Female	Basic with Ads	
14993	SUB114993	2026-04-03	India	South Asia	53	Male	Basic with Ads	

486 rows × 15 columns

```

In [49]: # From subscribers_df removing the rows where short_tenure = True
subscribers_df = subscribers_df[subscribers_df['short_tenure'] == False]

# Remove the flag short_tenure from subscribers_df
subscribers_df = subscribers_df.drop(['short_tenure'], axis=1)

subscribers_df

```

Out [49]:

	subscriber_id	signup_date	country	region	age	gender	plan_type	mc
0	SUB100000	2023-01-09	France	Europe	22	Female	Basic with Ads	
1	SUB100001	2020-02-05	United Kingdom	Europe	48	Male	Basic with Ads	
2	SUB100002	2022-04-15	Italy	Europe	35	Male	Standard	
3	SUB100003	2024-10-10	Spain	Europe	49	Female	Basic with Ads	
4	SUB100004	2020-04-12	United States	North America	36	Female	Basic with Ads	
...	...	...	...	...	...	...	...	...
14995	SUB114995	2023-06-14	Italy	Europe	40	Male	Standard	
14996	SUB114996	2025-11-29	Japan	East Asia	54	Female	Standard	
14997	SUB114997	2025-09-16	United States	North America	22	Male	Basic with Ads	
14998	SUB114998	2023-06-04	India	South Asia	19	Male	Basic with Ads	
14999	SUB114999	2023-08-06	Italy	Europe	16	Male	Premium	

14514 rows × 14 columns

```
In [50]: # subscribers country should have 20 different values.
country_count = len(subscribers_df['country'].value_counts())
if country_count == 20:
    print('country in subscribers.csv has 20 unique values')
else:
    print('country in subscribers.csv does not have 20 unique values')
```

country in subscribers.csv has 20 unique values

```
In [51]: # subscriber region should have 9 different values
count_region = len(subscribers_df['region'].value_counts())

if count_region == 9:
    print('region in subscribers.csv has 9 unique values')
else:
    print('region in subscribers.csv does not have 9 unique values')
```

region in subscribers.csv has 9 unique values

```
In [52]: # subscribers age should be between 13-85
if len(subscribers_df) == len(subscribers_df[subscribers_df.age.between(13, 85)]):
    print('age in subscribers.csv is between 13 and 85')
else:
    print('age in subscribers.csv is not between 13 and 85')
```

age in subscribers.csv is between 13 and 85

```
In [53]: # subscriber gender should be female/male/non-binary/undisclosed
gender_count = len(subscribers_df['gender'].value_counts())
```

```

if gender_count == 4:
    print('gender in subscribers.csv has 4 unique values')
else:
    print('gender in subscribers.csv does not have 4 unique values')

```

gender in subscribers.csv has 4 unique values

```

In [54]: # Subscribers plan type should be basic with ads/standard/ premium
plan_count = len(subscribers_df['plan_type'].value_counts())

if plan_count == 4:
    print('plan_type in subscribers.csv has 4 unique values')
else:
    print('plan_type in subscribers.csv does not have 4 unique values')

```

plan_type in subscribers.csv does not have 4 unique values

```

In [55]: # subscribers monthly_price_usd should be 6.99/ 15.49/ 22.99
prices_count = len(subscribers_df['monthly_price_usd'].value_counts())

if prices_count == 3:
    print('monthly_price_usd in subscribers.csv has 3 unique values')
else:
    print('monthly_price_usd in subscribers.csv does not have 3 unique va

```

monthly_price_usd in subscribers.csv has 3 unique values

```

In [56]: # subscribers payment method should be credit card/ debit card/ paypal/ m
payment_method_count = len(subscribers_df['payment_method'].value_counts(

if payment_method_count == 6:
    print('payment_method in subscribers.csv has 6 unique values')
else:
    print('payment_method in subscribers.csv does not have 6 unique value

```

payment_method in subscribers.csv has 6 unique values

-----titles.csv-----

```

In [57]: # primary_genre in titles should have 16 different values
genre_count = len(titles_df['primary_genre'].value_counts())

if genre_count == 16:
    print('primary_genre in titles.csv has 16 unique values')
else:
    print('primary_genre in titles.csv does not have 16 unique values')

```

primary_genre in titles.csv has 16 unique values

```

In [58]: # country in titles should have 20 different values
title_count = len(titles_df['country'].value_counts())

if title_count == 20:
    print('country in titles.csv has 20 unique values')
else:
    print('country in titles.csv does not have 20 unique values')

```

country in titles.csv has 20 unique values

```
In [59]: # language in titles should have 14 different values
language_count = len(titles_df['language'].value_counts())

if language_count == 14:
    print('language in titles.csv has 14 unique values')
else:
    print('language in titles.csv does not have 14 unique values')
```

language in titles.csv has 14 unique values

```
In [60]: # release year should be between 1969 and 2026 inclusive
if len(titles_df) == len(titles_df[titles_df.release_year.isin(range(1969, 2027))]):
    print('release_year in titles.csv is between 1969 and 2026')
else:
    print('release_year in titles.csv is not between 1969 and 2026')
```

release_year in titles.csv is between 1969 and 2026

```
In [61]: # Number of seasons should be 0 for movies
titles_df[(titles_df['type'] == 'Movie') & (titles_df['seasons'] != 0)]
```

```
Out[61]: title_id title_name type primary_genre country language release_year date_
```

0 rows x 21 columns

```
In [62]: # license_type should be original / exclusive license / non-exclusive license
licence_type_count = len(titles_df['license_type'].value_counts())

if licence_type_count == 4:
    print('license_type in titles.csv has 4 unique values')
else:
    print('license_type in titles.csv does not have 4 unique values')
```

license_type in titles.csv has 4 unique values

```
In [63]: # quality_score should be between 0 and 100 inclusive
is_valid_mask = titles_df['quality_score'].between(0, 100)

quality_score_validation = is_valid_mask.all()

if not quality_score_validation:
    quality_score_df = titles_df[~is_valid_mask]
    print(quality_score_df['quality_score'])
else:
    print("All quality scores are valid and between 0 and 100.")
```

All quality scores are valid and between 0 and 100.

```
In [64]: # popularity score in titles should be between 0 and 1 inclusive
is_valid_popularity = titles_df['popularity_score'].between(0, 1)

popularity_score_validation = is_valid_popularity.all()

if not popularity_score_validation:
    popularity_score_df = titles_df[~is_valid_popularity]
    print(popularity_score_df['popularity_score'])
else:
    print("All popularity scores are between 0 and 1 inclusive.")
```

```

12      12.11
19      2.18
23      2.27
24      1.23
27      1.89

```

```

...
8970    1.84
8979    4.23
8982    1.17
8998    6.18
8999    1.60

```

Name: popularity_score, Length: 1285, dtype: float64

-----reviews.csv-----

```

In [65]: # sentiment should be positive/neutral/negative
sentiment_count = len(reviews_df['sentiment'].value_counts())

if sentiment_count == 3:
    print('sentiment in reviews.csv has 3 unique values')
else:
    print('sentiment in reviews.csv does not have 3 unique values')

```

sentiment in reviews.csv has 3 unique values

-----ratings.csv-----

```

In [66]: # rating should be between 1-5 inclusive
is_valid_rating = ratings_df['rating'].between(1, 5)

rating_validation = is_valid_rating.all()

if not rating_validation:
    rating_df = ratings_df[~is_valid_rating]
    print(rating_df['rating'])
else:
    print("All ratings are valid and between 1-5 inclusive")

```

All ratings are valid and between 1-5 inclusive

-----watch_history.csv---

```

In [67]: # Confirm completion_pct roughly equals watch_duration_min + content_dura
watch_history_df['completion_pct'].equals(watch_history_df['watch_duratio
# Confirm that watch_duration_min is less than content_duration_min
watch_history_df['watch_duration_min'].lt(watch_history_df['content_durat

# Create a flag where these conditions are not followed
watch_history_df['watch_duration_min'].lt(watch_history_df['content_durat
watch_history_df['exceeds_content_len'] = watch_history_df['watch_duratio
watch_history_df['is_stat_outlier'] = watch_history_df['watch_duration_mi

```

```
watch_history_df['is_flagged'] = watch_history_df['watch_duration_min'].l
watch_history_df['is_flagged'].value_counts()
```

```
Out[67]: is_flagged
True      643795
False      436
Name: count, dtype: int64
```

```
In [68]: watch_history_df[watch_history_df['is_flagged'] == False]

# From watch_history_df remove enteries where is_flagged is False
watch_history_df[watch_history_df['is_flagged'] == True]

# Remove columns from watch_history_df
watch_history_df = watch_history_df.drop(['is_stat_outlier', 'exceeds_con
watch_history_df
```

```
Out[68]:
```

	watch_id	subscriber_id	title_id	watch_date	device	region	co
0	1	SUB107630	TTL205207	2024-01-01	Smart TV	East Asia	
1	2	SUB114773	TTL204889	2025-10-18	Mobile	South Asia	
2	3	SUB106013	TTL206882	2026-04-26	Mobile	Latin America	
3	4	SUB100939	TTL208849	2026-03-09	Mobile	North America	
4	5	SUB107223	TTL202529	2023-12-27	Tablet	North America	
...	...	...	...	...	...	...	...
649995	649996	SUB113946	TTL207486	2025-07-13	Mobile	Europe	
649996	649997	SUB108172	TTL200072	2026-05-22	Smart TV	Latin America	
649997	649998	SUB108092	TTL204011	2025-01-25	Streaming Stick	South Asia	
649998	649999	SUB114820	TTL204762	2026-04-13	Tablet	North America	
649999	650000	SUB101592	TTL206202	2022-10-16	Game Console	Latin America	

644231 rows × 10 columns

```
In [69]: # device in watch_history should be of 6 types
device_count = len(watch_history_df['device'].value_counts())

if device_count == 6:
    print('device in watch_history has 6 unique values')
else:
    print('device in watch_history does not have 6 unique values')
```

device in watch_history has 6 unique values

```
In [70]: # region in watch_history should be of 9 types
region_count = len(watch_history_df['region'].value_counts())

if region_count == 9:
    print('region in watch_history has 9 unique values')
else:
    print('region in watch_history does not have 9 unique values')
```

region in watch_history has 9 unique values

```
In [71]: # completion_pct should be between 0 and 100 inclusive# quality_score sho
is_valid_mask = watch_history_df['completion_pct'].between(0, 100)

completion_pct_validation = is_valid_mask.all()

if not completion_pct_validation:
    completion_pct_df = watch_history_df[~is_valid_mask]
    print(completion_pct_df['completion_pct'])
else:
    print("All completion_pct scores are valid and between 0 and 100.")
```

All completion_pct scores are valid and between 0 and 100.

-----watchlist.csv-----

```
In [72]: # Does every watchlist entry link to a real subscriber and a real title?
missing_fk(
    child_df = watchlist_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---

Total orphan records: 2166

Unique missing 'subscriber_id' keys: 482

Sample orphan records:

	watchlist_id	subscriber_id	title_id	added_date	watched
34	35	SUB104304	TTL203589	2026-04-08	False
42	43	SUB111332	TTL200208	2026-05-01	False
56	57	SUB101686	TTL208274	2026-04-05	False
140	141	SUB112784	TTL200086	2026-05-29	False
144	145	SUB101310	TTL202455	2026-03-08	True

Out [72]:

	watchlist_id	subscriber_id	title_id	added_date	watched
34	35	SUB104304	TTL203589	2026-04-08	False
42	43	SUB111332	TTL200208	2026-05-01	False
56	57	SUB101686	TTL208274	2026-04-05	False
140	141	SUB112784	TTL200086	2026-05-29	False
144	145	SUB101310	TTL202455	2026-03-08	True
...	...	...	...	...	...
64844	64845	SUB109237	TTL203327	2026-05-06	False
64857	64858	SUB114947	TTL202633	2026-04-03	True
64948	64949	SUB108248	TTL208893	2026-04-22	True
64965	64966	SUB110716	TTL200802	2026-05-01	True
64993	64994	SUB114600	TTL202790	2026-05-20	False

2166 rows × 5 columns

In [73]:

```
missing_fk(
    child_df = watchlist_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---

Total orphan records: 0

Unique missing 'title_id' keys: 0

Out [73]:

watchlist_id	subscriber_id	title_id	added_date	watched
--------------	---------------	----------	------------	---------

Running referential integrity checks again

----- watch_history.csv -

In [74]:

```
missing_fk(
    child_df = watch_history_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Watch History'
)
```

--- Foreign Key Integrity Check: Watch History ---

Total orphan records: 17901

Unique missing 'subscriber_id' keys: 486

Sample orphan records:

	watch_id	subscriber_id	title_id	watch_date	device	region
53	54	SUB104744	TTL202107	2026-05-07	Mobile	North America
55	56	SUB109335	TTL207473	2026-05-18	Tablet	North America
109	110	SUB111238	TTL207246	2026-05-16	Mobile	Europe
152	153	SUB102204	TTL205177	2026-05-22	Mobile	South Asia
183	184	SUB110250	TTL207167	2026-05-30	Mobile	Europe

	content_duration_min	watch_duration_min	completion_pct	completed
53	126	11.0	8.8	False
55	558	347.6	62.3	False
109	107	53.4	49.9	False
152	1536	1335.9	87.0	False
183	158	108.8	68.9	False

Out[74]:

	watch_id	subscriber_id	title_id	watch_date	device	region	com
53	54	SUB104744	TTL202107	2026-05-07	Mobile	North America	
55	56	SUB109335	TTL207473	2026-05-18	Tablet	North America	
109	110	SUB111238	TTL207246	2026-05-16	Mobile	Europe	
152	153	SUB102204	TTL205177	2026-05-22	Mobile	South Asia	
183	184	SUB110250	TTL207167	2026-05-30	Mobile	Europe	
...	...	...	...	...	...	...	...
649805	649806	SUB113141	TTL200841	2026-05-31	Laptop	North America	
649852	649853	SUB109083	TTL207146	2026-04-10	Smart TV	North America	
649878	649879	SUB112903	TTL203086	2026-05-12	Smart TV	Southeast Asia	
649880	649881	SUB109763	TTL203837	2026-05-23	Game Console	South Asia	
649894	649895	SUB102535	TTL201658	2026-05-23	Smart TV	East Asia	

17901 rows × 10 columns

In [75]: *# Removing the enteries from watch_history_df where subscriber_id from wa*
 watch_history_df = watch_history_df[watch_history_df['subscriber_id'].isn

```
missing_fk(
    child_df = watch_history_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
```

```
pk_col = 'subscriber_id',
df_name = 'Watch History'
)
```

--- Foreign Key Integrity Check: Watch History ---

Total orphan records: 0

Unique missing 'subscriber_id' keys: 0

Out [75]: **watch_id subscriber_id title_id watch_date device region content_duration_r**

```
In [76]: missing_fk(
    child_df = watch_history_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Watch History'
)
```

--- Foreign Key Integrity Check: Watch History ---

Total orphan records: 0

Unique missing 'title_id' keys: 0

Out [76]: **watch_id subscriber_id title_id watch_date device region content_duration_r**

-----ratings.csv-----

```
In [77]: missing_fk(
    child_df = ratings_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Ratings'
)
```

--- Foreign Key Integrity Check: Ratings ---

Total orphan records: 4127

Unique missing 'subscriber_id' keys: 486

Sample orphan records:

	rating_id	subscriber_id	title_id	rating	rating_date
12	13	SUB104921	TTL202344	4	2022-09-20
25	26	SUB113673	TTL201316	2	2025-05-21
56	57	SUB113900	TTL205245	4	2026-05-06
156	157	SUB110420	TTL204871	4	2026-05-16
206	207	SUB102204	TTL207585	4	2026-02-18

Out [77]:

	rating_id	subscriber_id	title_id	rating	rating_date
	12	13	SUB104921	TTL202344	4 2022-09-20
	25	26	SUB113673	TTL201316	2 2025-05-21
	56	57	SUB113900	TTL205245	4 2026-05-06
	156	157	SUB110420	TTL204871	4 2026-05-16
	206	207	SUB102204	TTL207585	4 2026-02-18
	...	...	...	...	...
	129782	129783	SUB102793	TTL203801	4 2026-03-08
	129906	129907	SUB110220	TTL205519	4 2019-08-31
	129946	129947	SUB105078	TTL207759	4 2021-12-05
	129974	129975	SUB106402	TTL202083	2 2025-11-18
	129992	129993	SUB111951	TTL208256	4 2026-01-25

4127 rows × 5 columns

```
In [78]: # From ratings_df, removing the entries where subscriber_id from ratings_
ratings_df = ratings_df[ratings_df['subscriber_id'].isin(subscribers_df['
missing_fk(
    child_df = ratings_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Ratings'
)]
```

```
--- Foreign Key Integrity Check: Ratings ---
Total orphan records: 0
Unique missing 'subscriber_id' keys: 0
```

```
Out [78]: rating_id subscriber_id title_id rating rating_date
```

```
In [79]: missing_fk(
    child_df = ratings_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Ratings'
)]
```

```
--- Foreign Key Integrity Check: Ratings ---
Total orphan records: 0
Unique missing 'title_id' keys: 0
```

```
Out [79]: rating_id subscriber_id title_id rating rating_date
```

```
-----reviews.csv-----
-----
```

```
In [80]: missing_fk(
    child_df = reviews_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Reviews'
)
```

--- Foreign Key Integrity Check: Reviews ---

Total orphan records: 3593

Unique missing 'subscriber_id' keys: 486

Sample orphan records:

	review_id	subscriber_id	title_id	\
69	70	SUB107134	TTL200365	
83	84	SUB111323	TTL204417	
85	86	SUB102683	TTL201792	
136	137	SUB102384	TTL208849	
199	200	SUB112071	TTL201234	

	review_text	sentiment	\
69	Average. Worth it if you like the genre.	Neutral	
83	Average. Worth it if you like the genre.	Neutral	
85	The writing is sharp and the pacing is perfect.	Positive	
136	Watchable, though the ending felt rushed.	Neutral	
199	Couldn't stop watching – binged it in a weekend.	Positive	

	helpful_votes	review_date
69	2	2026-05-28
83	1	2026-01-19
85	9	2024-08-06
136	6	2025-12-29
199	12	2026-05-28

Out [80]:

	review_id	subscriber_id	title_id	review_text	sentiment	helpful_
69	70	SUB107134	TTL200365	Average. Worth it if you like the genre.	Neutral	
83	84	SUB111323	TTL204417	Average. Worth it if you like the genre.	Neutral	
85	86	SUB102683	TTL201792	The writing is sharp and the pacing is perfect.	Positive	
136	137	SUB102384	TTL208849	Watchable, though the ending felt rushed.	Neutral	
199	200	SUB112071	TTL201234	Couldn't stop watching — binged it in a weekend.	Positive	
...	...	...	...	...	...	...
109820	109821	SUB114947	TTL205371	Decent watch but nothing groundbreaking.	Neutral	
109842	109843	SUB101121	TTL205903	Beautifully shot and emotionally resonant.	Positive	
109863	109864	SUB100869	TTL205704	Stellar cast and a satisfying ending.	Positive	
109961	109962	SUB113900	TTL203186	One of the best things I've streamed this year.	Positive	
109992	109993	SUB105416	TTL201910	Brilliant performances and a tight script.	Positive	

3593 rows x 7 columns

```
In [81]: # Removing the entries from reviews_df where subscriber_id from reviews_d
reviews_df = reviews_df[reviews_df['subscriber_id'].isin(subscribers_df['

missing_fk(
    child_df = reviews_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Reviews'
)
```

--- Foreign Key Integrity Check: Reviews ---

Total orphan records: 0

Unique missing 'subscriber_id' keys: 0

Out[81]:

review_id	subscriber_id	title_id	review_text	sentiment	helpful_votes	review_c
-----------	---------------	----------	-------------	-----------	---------------	----------

```
In [82]: missing_fk(
    child_df = reviews_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Reviews'
)
```

--- Foreign Key Integrity Check: Reviews ---

Total orphan records: 0

Unique missing 'title_id' keys: 0

Out[82]:

review_id	subscriber_id	title_id	review_text	sentiment	helpful_votes	review_c
-----------	---------------	----------	-------------	-----------	---------------	----------

-----watchlist.csv-----

```
In [83]: missing_fk(
    child_df = watchlist_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---

Total orphan records: 2166

Unique missing 'subscriber_id' keys: 482

Sample orphan records:

	watchlist_id	subscriber_id	title_id	added_date	watched
34	35	SUB104304	TTL203589	2026-04-08	False
42	43	SUB111332	TTL200208	2026-05-01	False
56	57	SUB101686	TTL208274	2026-04-05	False
140	141	SUB112784	TTL200086	2026-05-29	False
144	145	SUB101310	TTL202455	2026-03-08	True

Out [83]:

	watchlist_id	subscriber_id	title_id	added_date	watched
34	35	SUB104304	TTL203589	2026-04-08	False
42	43	SUB111332	TTL200208	2026-05-01	False
56	57	SUB101686	TTL208274	2026-04-05	False
140	141	SUB112784	TTL200086	2026-05-29	False
144	145	SUB101310	TTL202455	2026-03-08	True
...	...	...	...	...	...
64844	64845	SUB109237	TTL203327	2026-05-06	False
64857	64858	SUB114947	TTL202633	2026-04-03	True
64948	64949	SUB108248	TTL208893	2026-04-22	True
64965	64966	SUB110716	TTL200802	2026-05-01	True
64993	64994	SUB114600	TTL202790	2026-05-20	False

2166 rows × 5 columns

In [84]: *# removing the entries from watchlist_df where subscriber_id from watchli*
 watchlist_df = watchlist_df[watchlist_df['subscriber_id'].isin(subscriber

```
missing_fk(
    child_df = watchlist_df,
    parent_df = subscribers_df,
    fk_col = 'subscriber_id',
    pk_col = 'subscriber_id',
    df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---
 Total orphan records: 0
 Unique missing 'subscriber_id' keys: 0

Out [84]:

watchlist_id	subscriber_id	title_id	added_date	watched
--------------	---------------	----------	------------	---------

In [85]:

```
missing_fk(
    child_df = watchlist_df,
    parent_df = titles_df,
    fk_col = 'title_id',
    pk_col = 'title_id',
    df_name = 'Watchlist'
)
```

--- Foreign Key Integrity Check: Watchlist ---
 Total orphan records: 0
 Unique missing 'title_id' keys: 0

Out [85]:

watchlist_id	subscriber_id	title_id	added_date	watched
--------------	---------------	----------	------------	---------

In [86]: *# Storing all the cleaned dataframes into new csv file in a new folder na*
import os

```
# # 1. Define the output folder path
# output_dir = "cleaned_data"
# os.makedirs(output_dir, exist_ok=True)

# # 2. Map file names to your cleaned DataFrame variables
# export_tables = {
#     "ratings.csv": ratings_df,
#     "reviews.csv": reviews_df,
#     "watchlist.csv": watchlist_df,
#     "watch_history.csv": watch_history_df,
#     "subscribers.csv": subscribers_df,
#     "titles.csv": titles_df
# }

# # 3. Export all DataFrames to CSV in the new folder
# for filename, df in export_tables.items():
#     file_path = os.path.join(output_dir, filename)
#     df.to_csv(file_path, index=False)
#     print(f" Saved: {file_path} ({len(df):,} rows)")

# print(
#     f"\n Successfully exported all {len(export_tables)} cleaned CSV fil
# )
```

Data Quality & Profiling Report: Phase 1

Data Cleaning

Executive Summary

A comprehensive data profiling and quality check was conducted across six core datasets (ratings , reviews , subscribers , titles , watch_history , and watchlist). Primary key integrity was confirmed with zero primary key duplicates across all tables. Missing values are isolated to conditional business attributes (churn_date and license_expiry). Standard date, boolean, and numeric casting transformations were applied to ensure clean schema consistency across all entities.

Dataset Volume & Schema Overview

Dataset	Total Records	Total Columns	Primary Key	Key Foreign Keys	Mis Su
ratings.csv	130,000	5	rating_id	subscriber_id , title_id	0 ((
reviews.csv	110,000	7	review_id	subscriber_id , title_id	0 ((
subscribers.csv	15,000	14	subscriber_id	None	ch 11,1

Dataset	Total Records	Total Columns	Primary Key	Key Foreign Keys	Mis Su
titles.csv	9,000	21	title_id	None	li 1,90
watch_history.csv	650,000	10	watch_id	subscriber_id , title_id	0 (0
watchlist.csv	65,000	5	watchlist_id	subscriber_id , title_id	0 (0

Detailed Data Quality & Integrity Findings

1. Primary Key & Duplicate Analysis

- **Primary Keys:** All primary keys (rating_id , review_id , subscriber_id , title_id , watch_id , watchlist_id) are 100% unique, verifying that entity constraints are strictly met.
- **Non-Key Cardinality & Duplication:**
 - High duplicate counts across user/content interaction keys confirm expected one-to-many relationships (e.g., subscriber_id in watch_history : 635,002 duplicates; title_id in ratings : 121,001 duplicates).
 - **titles.csv Title Name Duplicates:** 53 duplicate entries exist in title_name (representing multi-season entries, re-releases, or localized variations sharing identical names).

2. Missing Value Profiling

- Structural missingness is confined strictly to two temporal attributes dependent on business logic:
 - **subscribers.csv (churn_date):** 74.66% missing (11,199 records). These nulls represent active subscribers who have not churned (supported by is_active == True).
 - **titles.csv (license_expiry):** 21.84% missing (1,966 records). These represent owned/original content or perpetual licenses with no expiration date.
- All other tables (ratings , reviews , watch_history , watchlist) contain **zero null values** across all columns.

Applied Transformations & Type Corrections

Date & Timestamp Standardizations

Converted all raw string/object representations into standardized datetime64[ns] datatypes:

- **ratings_df** : rating_date
- **reviews_df** : review_date
- **subscribers_df** : signup_date , churn_date
- **titles_df** : date_added , license_expiry
- **watch_history_df** : watch_date
- **watchlist_df** : added_date

Additional Attribute Conversions

- **titles_df** : Cast **is_original** from object to **bool** .
- **titles_df** : Cast **license_cost_usd** from integer to **float64** for financial consistency.

In [0]:

In [0]: